

Profiling Large Language Model Trainings on the Grace Hopper Superchip using Nsight Systems

Karin Sevegnani, Senior Solutions Architect, NVAITC UK

IEEE Cluster, September 2nd 2025

About Me

Karin Sevegnani – ksevegnani@nvidia.com

- Senior Solutions Architect @ NVIDIA
  - Supporting Higher Education and Research through collaborations
  - Lead of **Nvidia AI Technology Center (NVAITC) UK**
- PhD in Conversational AI from Edinburgh Centre for Robotics (University of Edinburgh and Heriot-Watt University)
- Expertise in **Natural Language Processing**



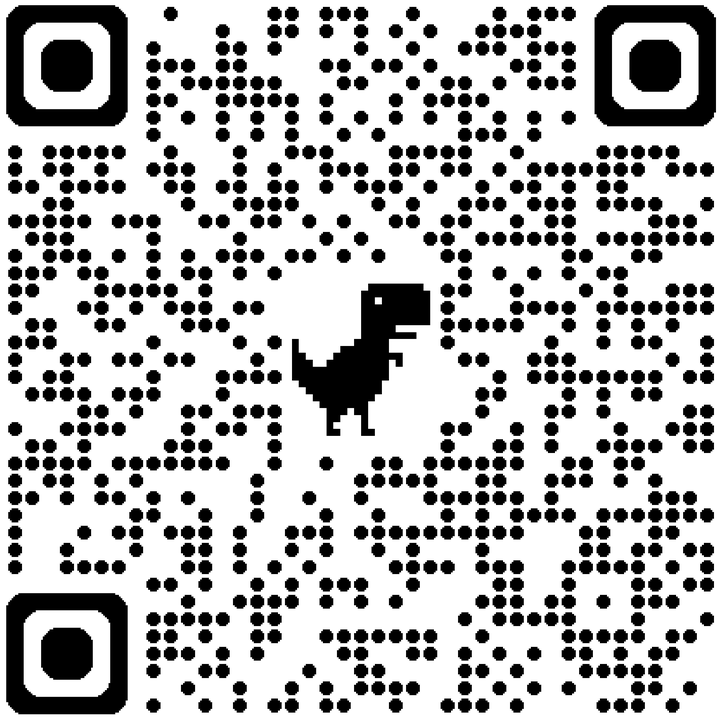


Agenda

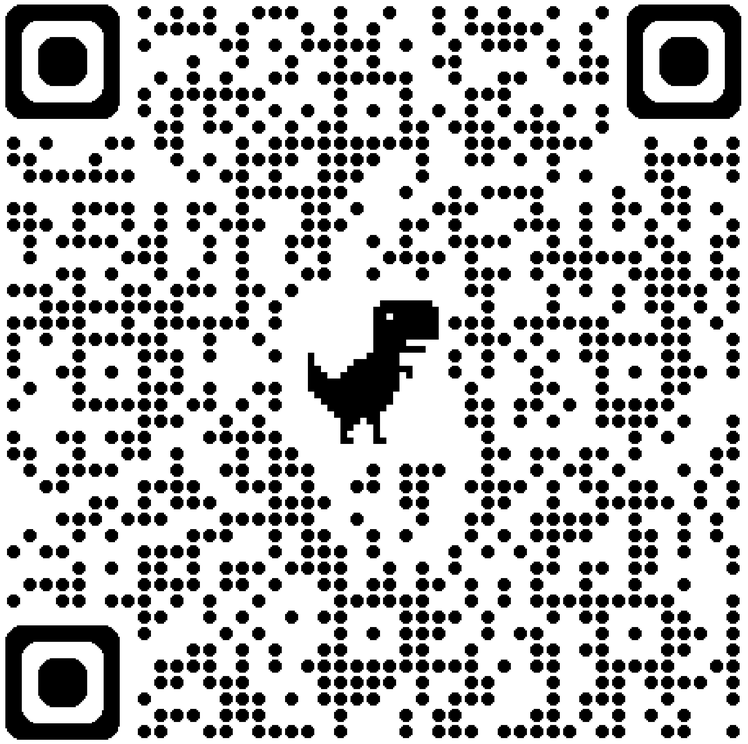
- Introduction
- Grace Hopper Superchip for LLM Training
- Profiling of an LLM using Nsight Systems
- LLM Profiling and Optimization on GH
- Best Practices for Optimization

Blogposts on Materials

Future References



Blog 1: Profiling LLM Training Workflows on NVIDIA Grace Hopper

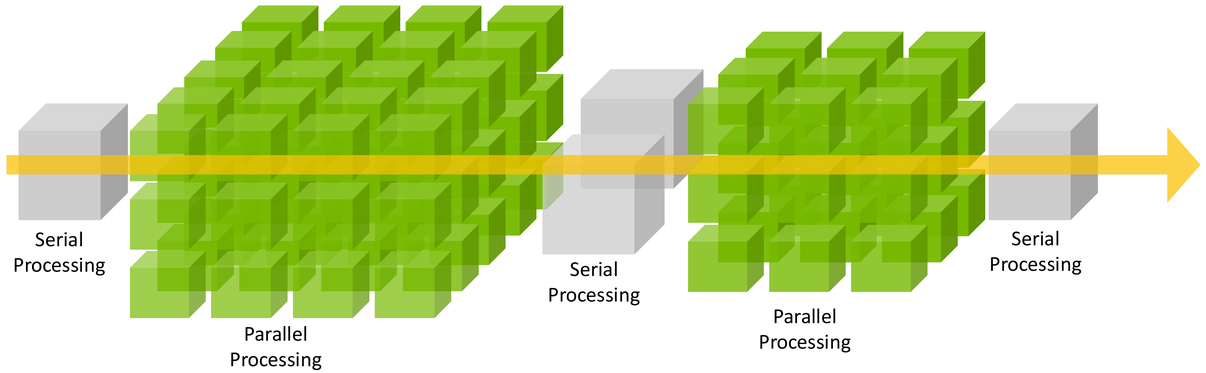


Blog 2: Advanced Optimization Strategies for LLM Training on NVIDIA Grace Hopper

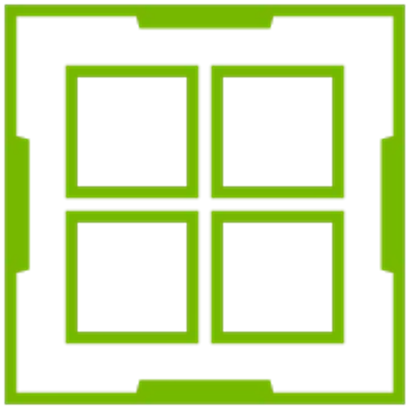
Modern Applications Integrate Sequential and Parallel Computing

Both CPU and GPU are Critical for Accelerated Compute

Modern Application Workflows



CPU



Excels at Serial Processing

GPU

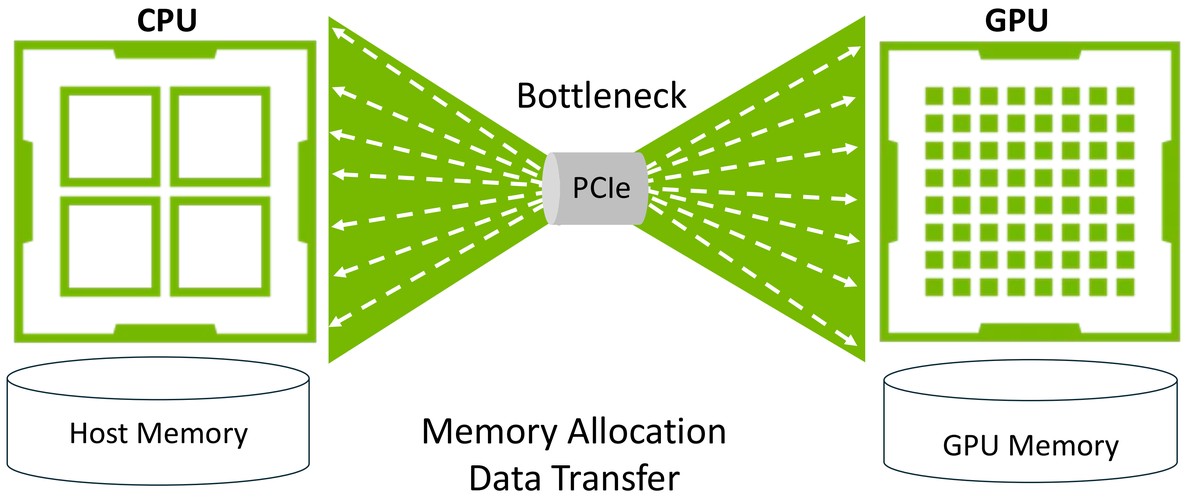


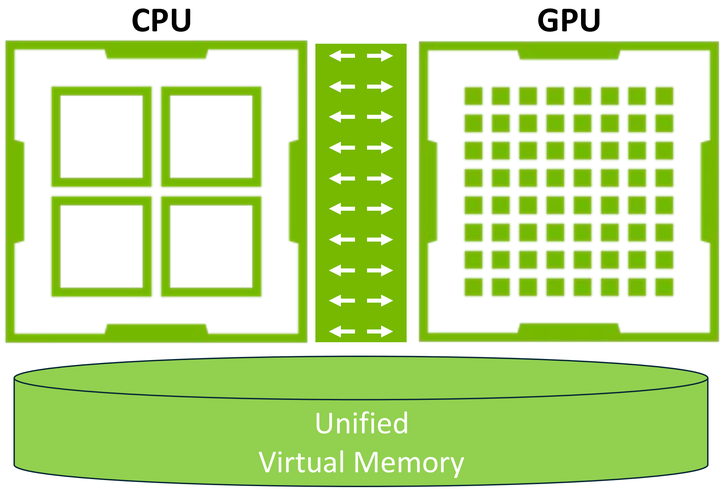
Excels at Parallel Processing

To further accelerate compute, and make it accessible to every workload we need to address CPU bottlenecks

Challenges with Traditional Accelerated Systems

Bridging the GAP: Unifying CPU and GPU Memory





- ❑ PCIe bottlenecks CPU-GPU communication
- ❑ GPU can't access CPU memory directly
- ❑ Complex Programmability
- ❑ Low CPU Performance per Watt

Legacy PCIe Architecture

We need a new chip architecture:

- ✓ Accelerate CPU-GPU data transfers
- ✓ Coherent CPU and GPU cache memory
- ✓ Simplify Programmability
- ✓ Deliver more performance per Watt

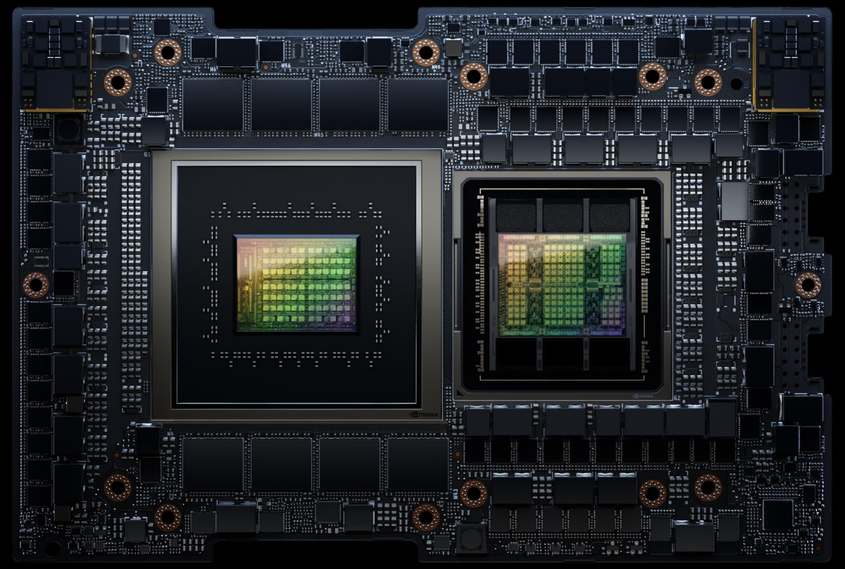
NVIDIA GH200 Grace Hopper Superchip

Built for the New Era of Accelerated Computing and Generative AI

Most versatile compute
Best performance across CPU, GPU or memory intensive applications

Easy to deploy and scale out
1 CPU:1 GPU node simple to manage and schedule for HPC, enterprise, and cloud

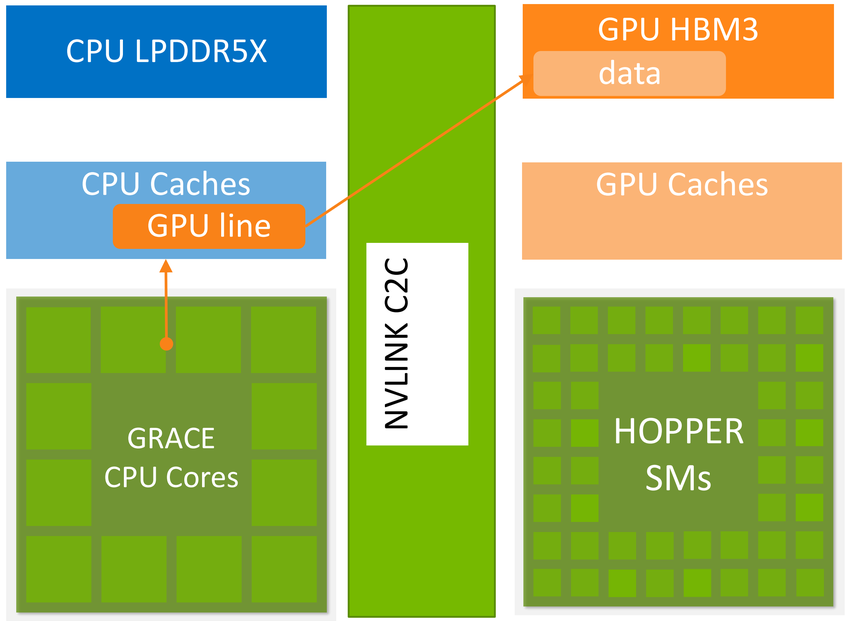
Best Perf/TCO for diverse workloads
Maximize data center utilization and power efficiency



900GB/s NVLink-C2C | 624GB High-Speed Memory
4 PF AI Perf | 72 Arm Cores

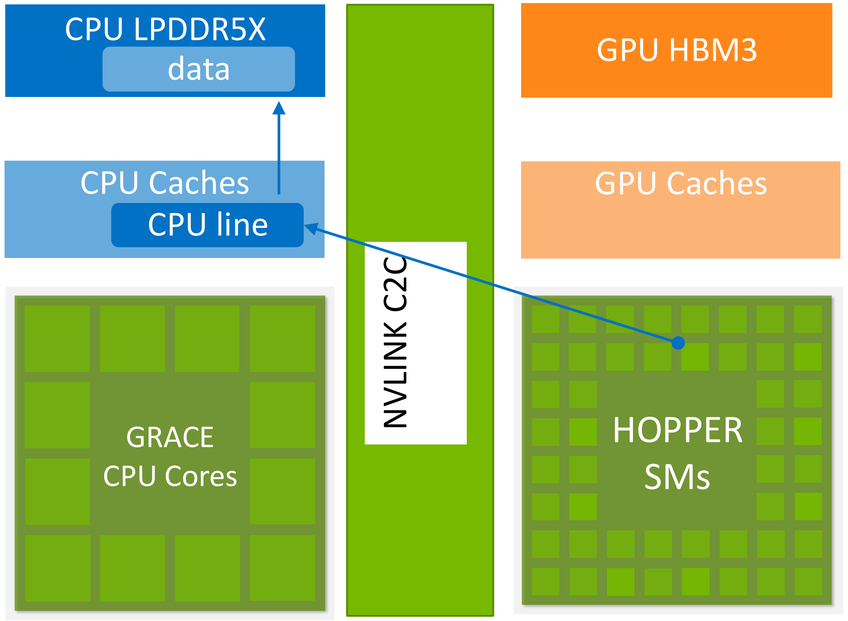
Global Access to All Data

Cache-Coherent Access Via NVLink C2C from Either Processor to Either Physical Memory



Grace directly reading Hopper's memory

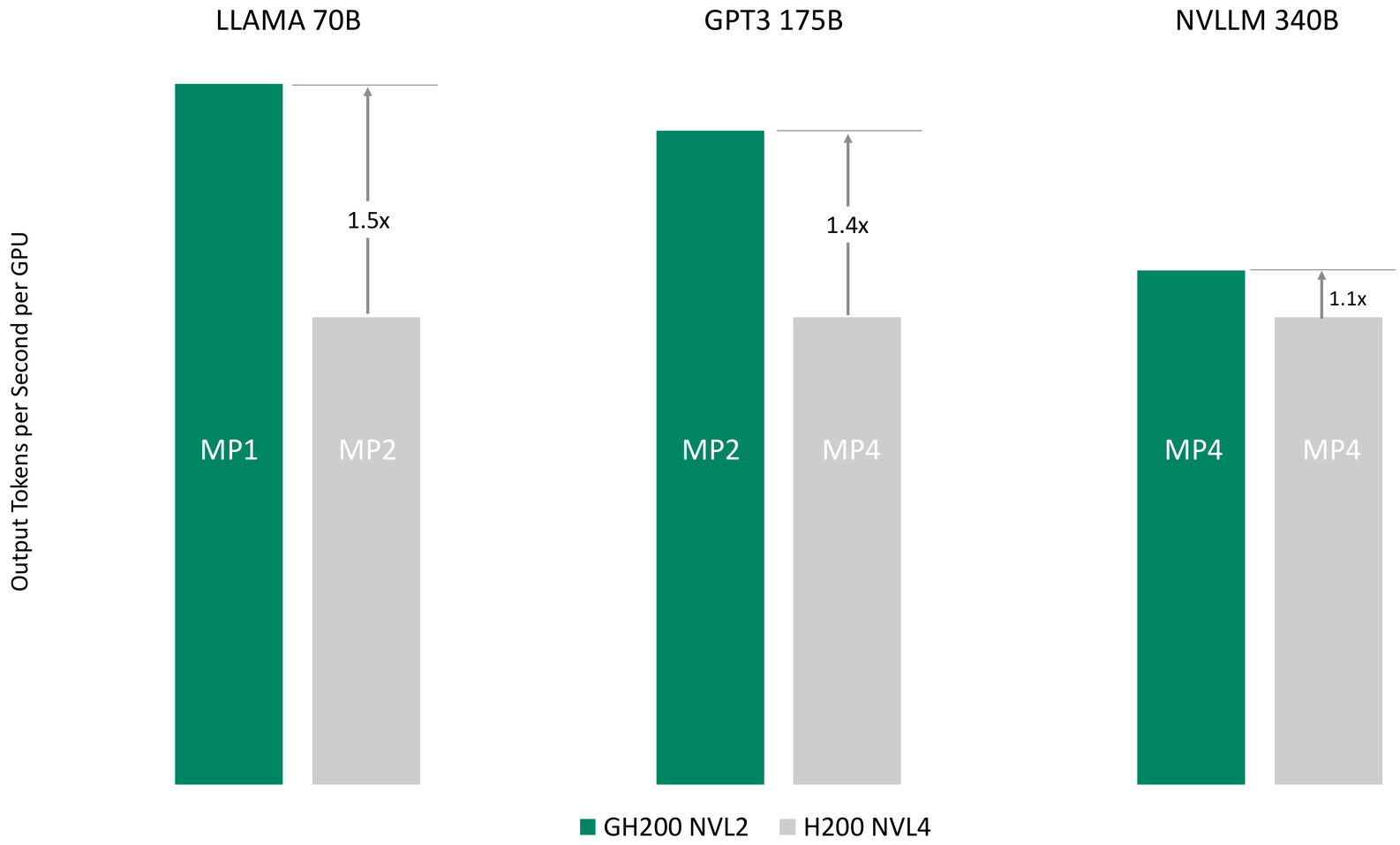
CPU fetches GPU data into CPU L3 cache
Cache remains **coherent** with GPU memory
Changes to GPU memory **evict** cache line



Hopper directly reading Grace's memory

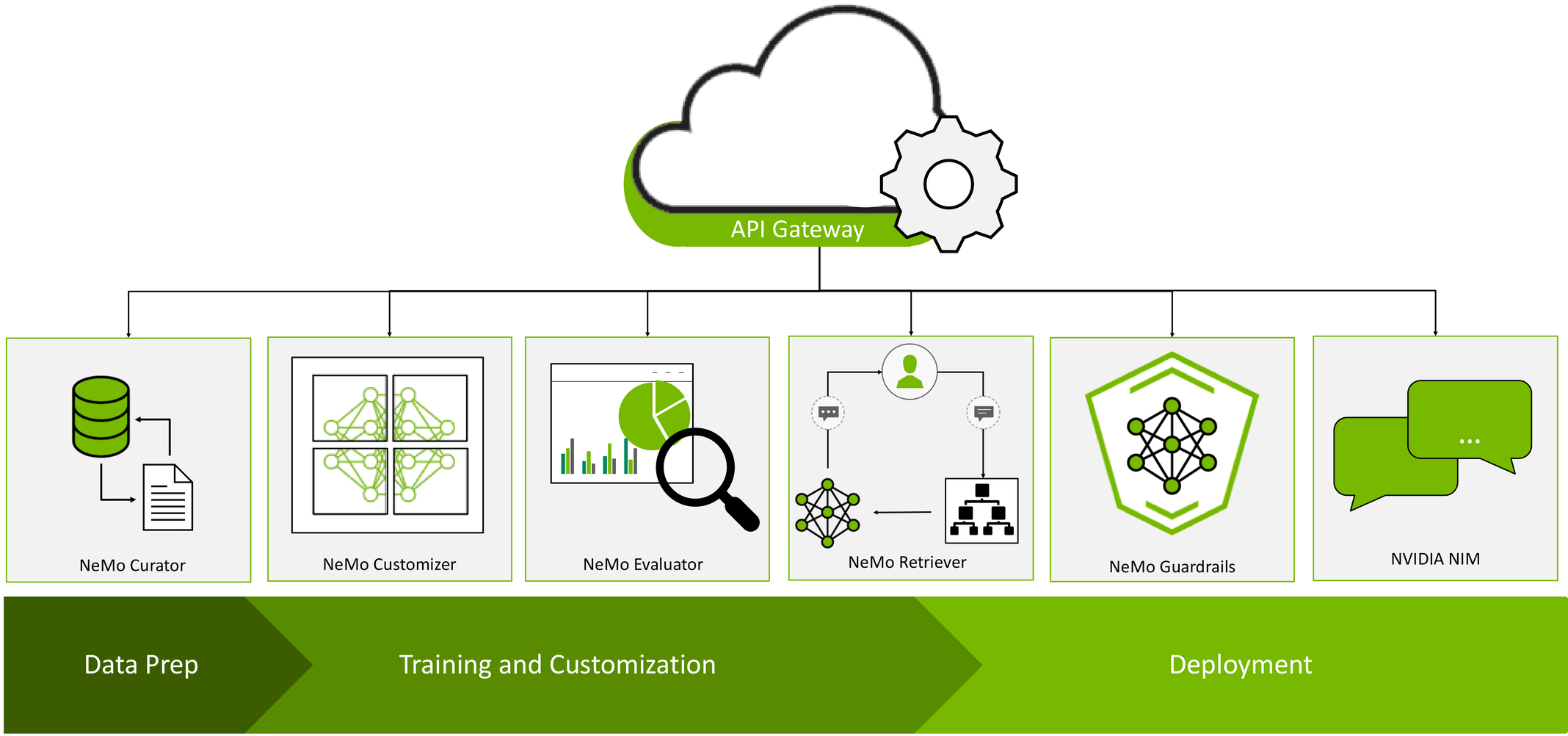
GPU loads CPU data via CPU L3 cache
CPU and GPU **can both hit** on cached data
Changes to CPU memory **update** cache line

Best Performance for Models Up To 175B



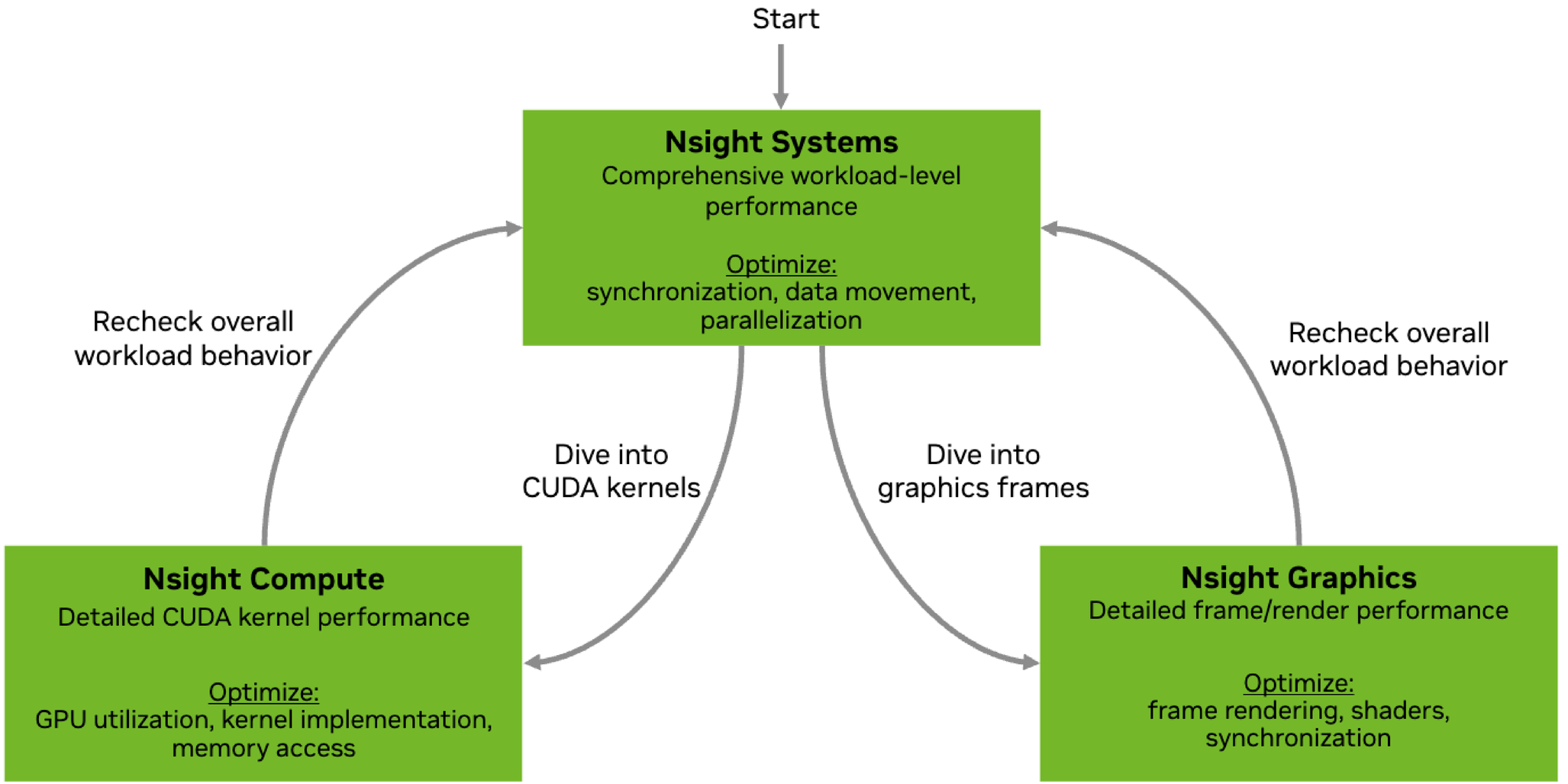
MP: Model Parallel
ISL/OSL 2048/128
GH200 NVL2: 700W+PowerSteering | H200 NVL4 : 600W

NeMo Framework



Profiling

Nsight Systems



Nsight Systems

System Profiler

Key Features

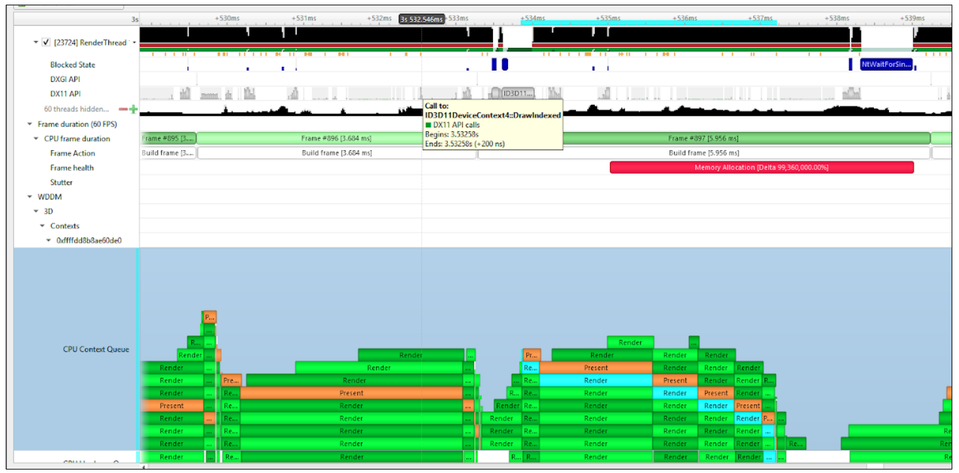
- System-wide application algorithm tuning
- Multi-process tree support
- Locate optimization opportunities
- Visualize millions of events on a very fast GUI timeline
- Or gaps of unused CPU and GPU time
- Balance your workload across multiple CPUs and GPUs
- CPU algorithms, utilization and thread state
  GPU streams, kernels, memory transfers, etc
- Command Line, Standalone, IDE Integration

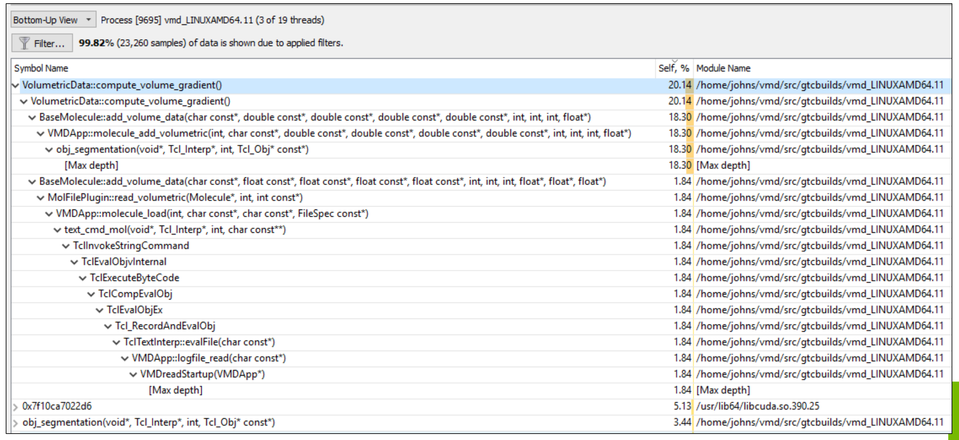
OS: Linux (x86, Power, Arm SBSA, Tegra), Windows, MacOSX (host)

GPUs: Pascal+

Docs/product: https://developer.nvidia.com/nsight-systems







Profiling

Nsight Command Line Interface

- The Nsight Systems CLI provides several commands. See `nsys --help`
- Basic profiling session:
  - `nsys profile`
- Interactive sessions (scriptable):
  - `nsys launch | start | stop | cancel`
  - `nsys session list`
  - `nsys status | shutdown`
- Statistics and export:
  - `nsys stats`
  - `nsys export` (export sqlite, hdf, text, json, info, arrow, arrowdir, parquetdir)
- Analysis:
  - `nsys analyze`
  - `nsys recipe`



Profiling

Nsight CLI Profiling Switches

- API tracing
  - `--t`, `--trace=cuda, nvtx, osrt, cudnn, cublas`
- Summary statistics (profile output on command line)
  - `--stats=[true|false]`
- Report file name
  - `-o`, `--output=report#` (patterns for hostname, PID %p and environment variables %q{ENV_VAR})
- Overwrite existing report
  - `-f`, `--force-overwrite=[true|false]`
- Use **`nsys profile --help`** for a list of available options

NVIDIA Tools Extensions (NVTX)

C, C++, Python

Instrument application source code

- Header-only library

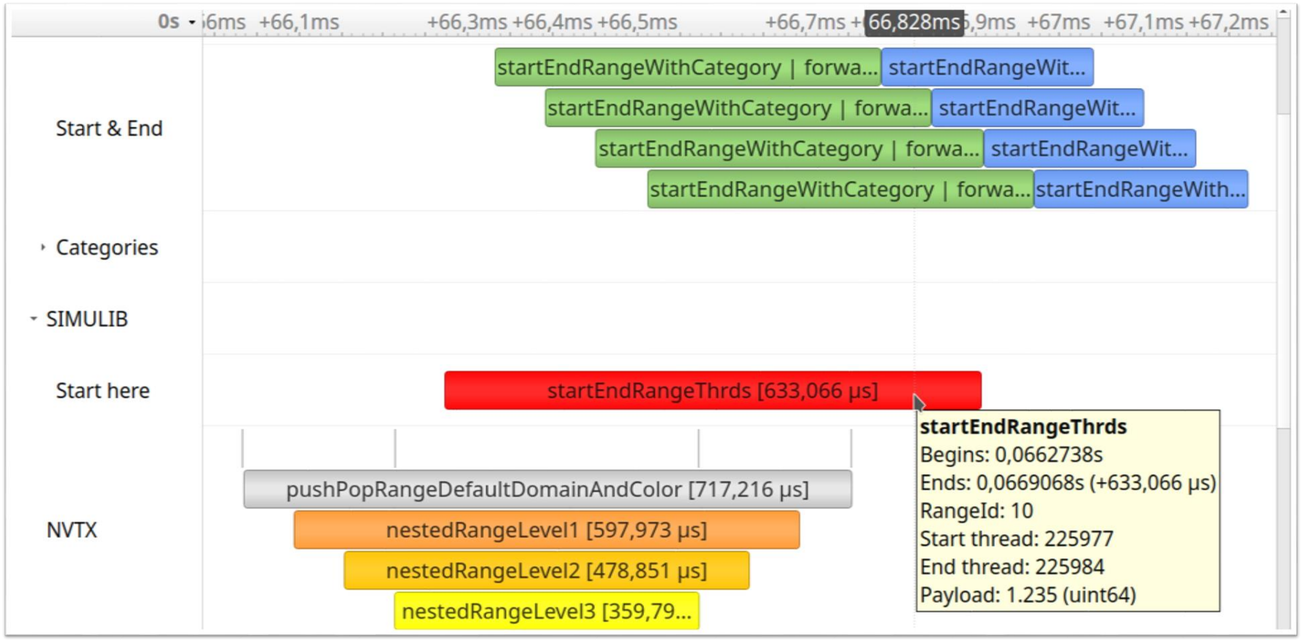
```
#include <nvtx3/nvToolsExt.h>
```

- **Marker**

```
nvtxMark("Point in time");
```

- **Push-Pop range**

```
nvtxRangePush(“Perfectly nested range");
// Do something interesting in the range.
nvtxRangePop();
```



- **Start-End range**

```
nvtxRangeId_t rangeId = nvtxRangeStart("This is a start/end range");
// Somewhere else in the code, not necessarily same thread as range start call
nvtxRangeEnd(rangeId);
```

- Provides concepts like domains, categories, colors, registered strings, and extended payloads (with user-defined layout)
- Sources: https://github.com/NVIDIA/NVTX/tree/release-v3/
- Enable Nsight Systems NVTX collection with --**trace nvtx** (enabled by default)

Experiments

Profiling

Set up - Machine

- **Machine**:
  - GH200 machine: 1 node, 4 x NVIDIA GH200 Grace Hopper Superchip
  - Power capping: 660Watts

- **Current hardware**:
  - Grace CPU 72 cores with 120GB memory
  - Hopper GPU with 96GB HMB3
  - Slingshot 11, each node has 4 x Cassini NICs, each NIC operating at 200Gbps

- SLURM job scheduler
- Singularity containers

Profiling

Set up - Environment

- Set up the container
  - Pull locally the internal NeMo for GH singularity image
  - Grab one node in interactive mode

```
salloc -n 1 -N 1 -p gh -t 2:00:00
```

  - Run the container inside the interactive node

```
singularity run --nv nemo-nightly.sif
```

- Download your model
  - LLama3-8B
- Download your dataset
  - Databricks-dolly-15k
- Download your codebase
  - NeMo FW



Data, Model and FW are already on the system:
/projects/t5c/public/

Profiling

Set up - Task

- SFT a 8B parameter model with LoRA, on a single GPU
- Following this Playbook: https://docs.nvidia.com/nemo-framework/user-guide/24.07/playbooks/llama2sft.html#step-2-data-preprocessing

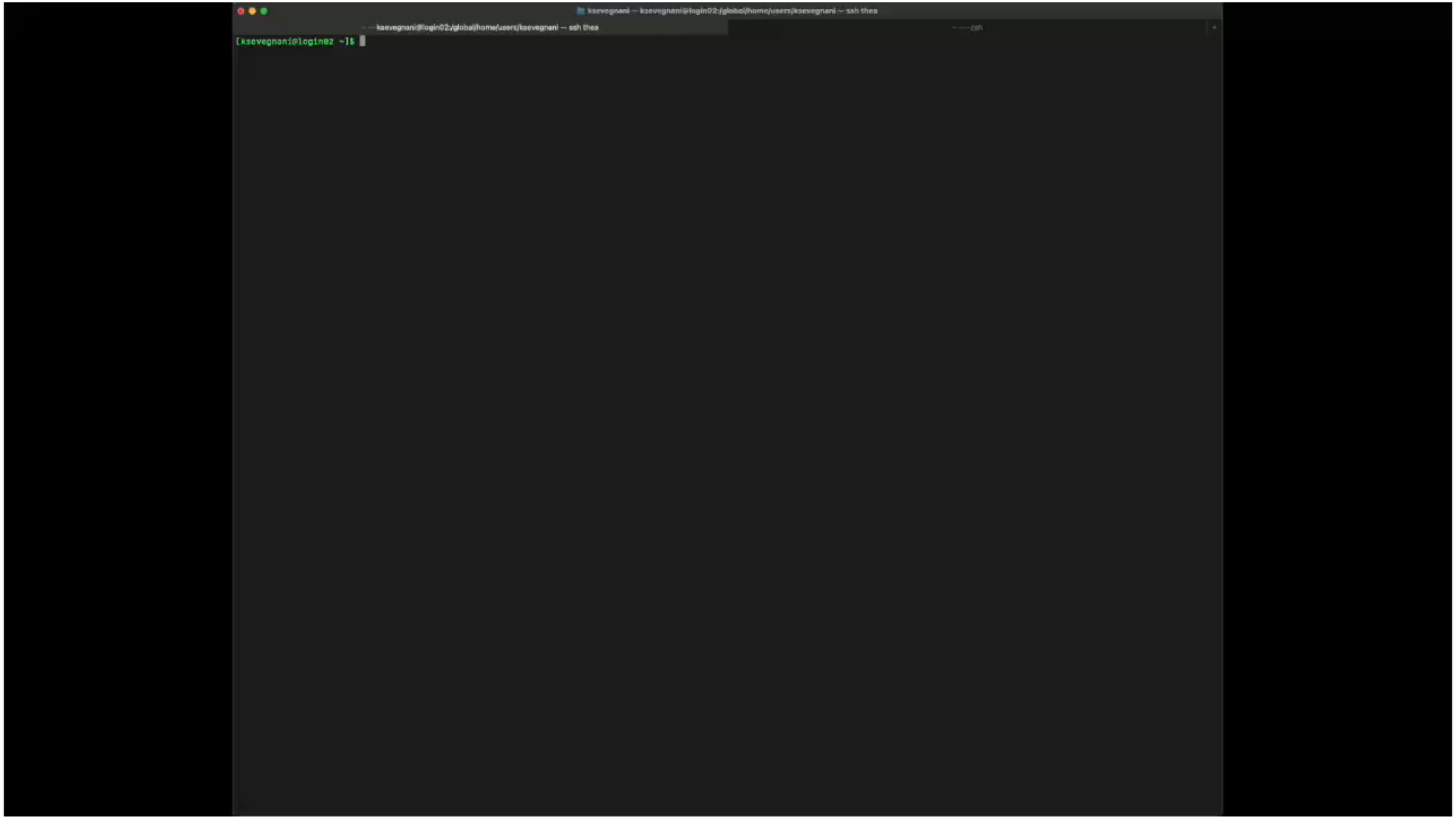
- Why LoRA?

- What do we want to monitor while profiling?

```
  nsys profile -y 360 -d 720 --trace=cuda,cudnn,cublas,osrt,nvtx --event-sample=system-wide -w true
-c cudaProfilerApi -o sft_llama2_7B python …
```

Profiling

Video of job set up and run

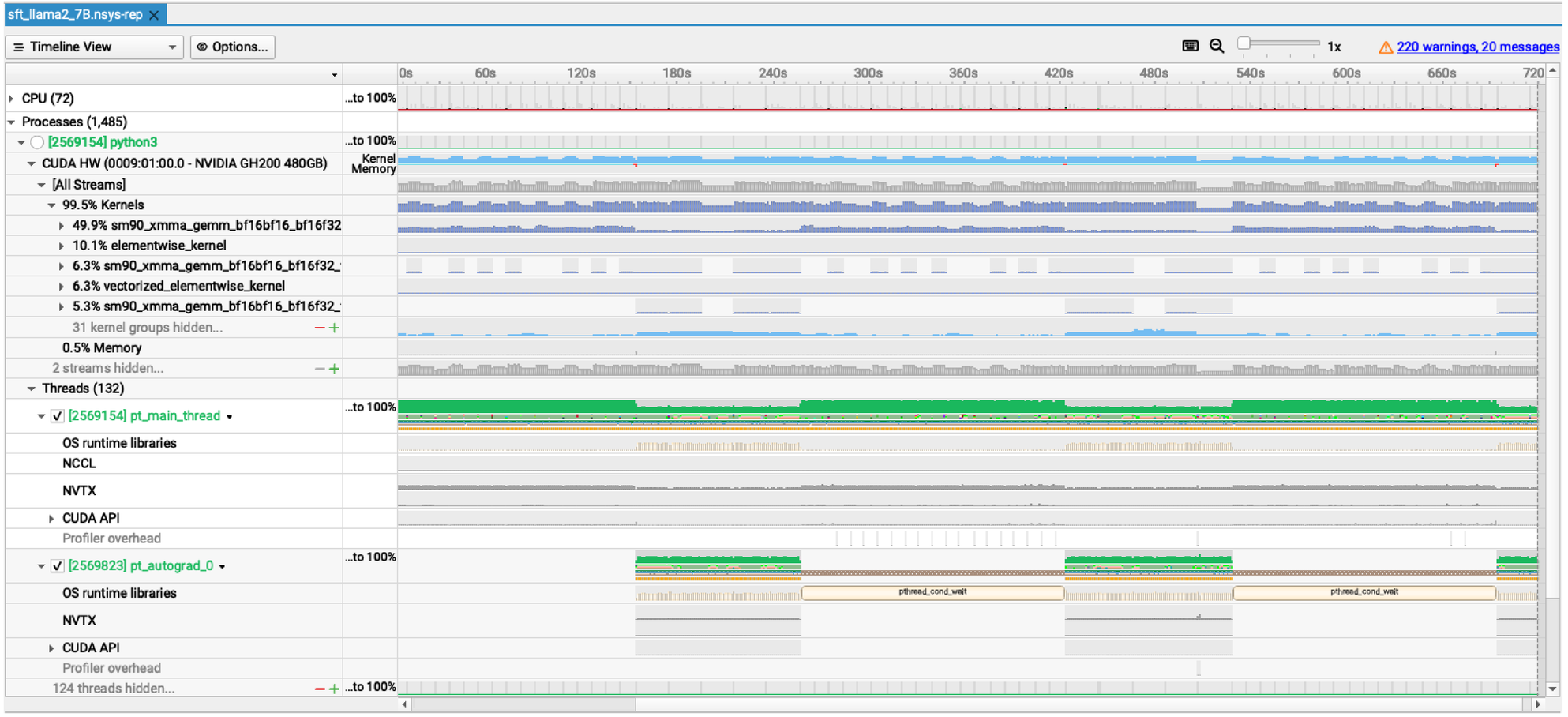


Profiling

Nsight Systems

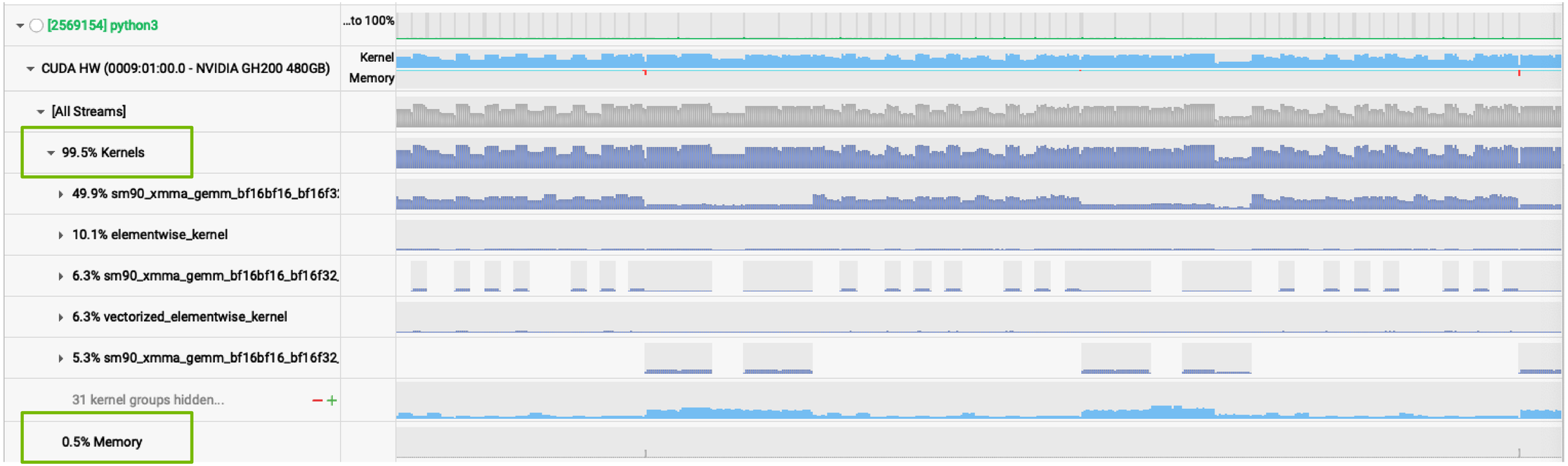
- Copy your `.nsys-rep` file over to your local machine
- Drag and drop it on Nsight Systems

How can you interpret this?



Profiling

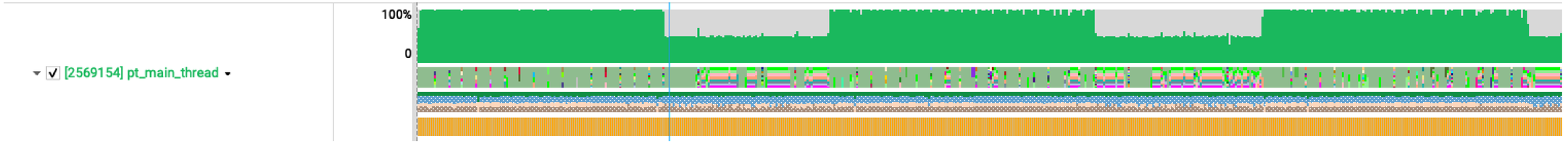
Nsight Systems – Pt 1



- 99.5% Kernels vs 0.5% Memory: the training workflow is **compute-bound** and not **memory-bound**
  - **Compute- bound**: a process is considered compute-bound when its performance is primarily limited by the speed of the processor (CPU or GPU).
  - **Memory-bound**: a process is considered memory-bound when its performance is primarily limited by the speed of the memory access (reading and writing data) rather than by computation.

Profiling

Nsight Systems – Pt 2



Interpretation:

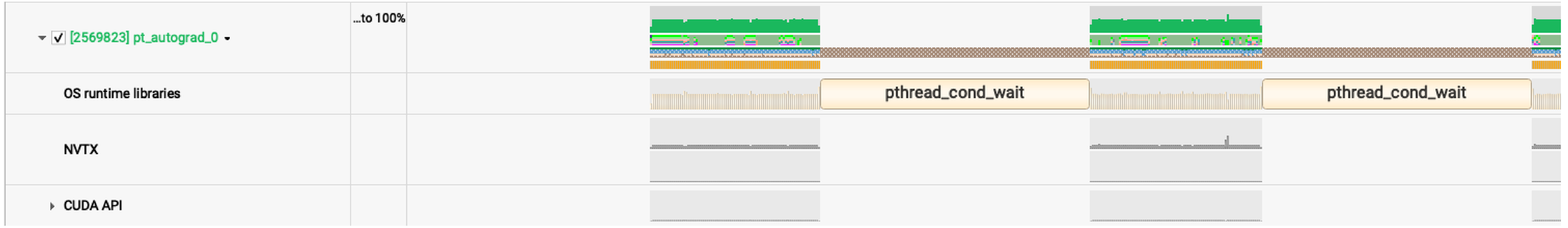
- Green bar: GPU activity
- Grey spaces: idle GPU
- Colorful blocks below: CPU threads
- Orange bars: CUDA memory transfers

Interpretation:

- High GPU Utilization, but understand idle periods: delays in data processing on the CPU? Insufficient overlap of computation and communication? Synchronization issues between kernels and processes?
- Sparse orange bars: memory transfers are not dominating the execution time (ideal for dl workloads, as most data should reside on the GPU)

Profiling

Nsight Systems – Pt 3



- **Autograd** engine in PyTorch
  - the core component responsible for automatic differentiation
  - computes gradients for tensors during backpropagation

Interpretation:

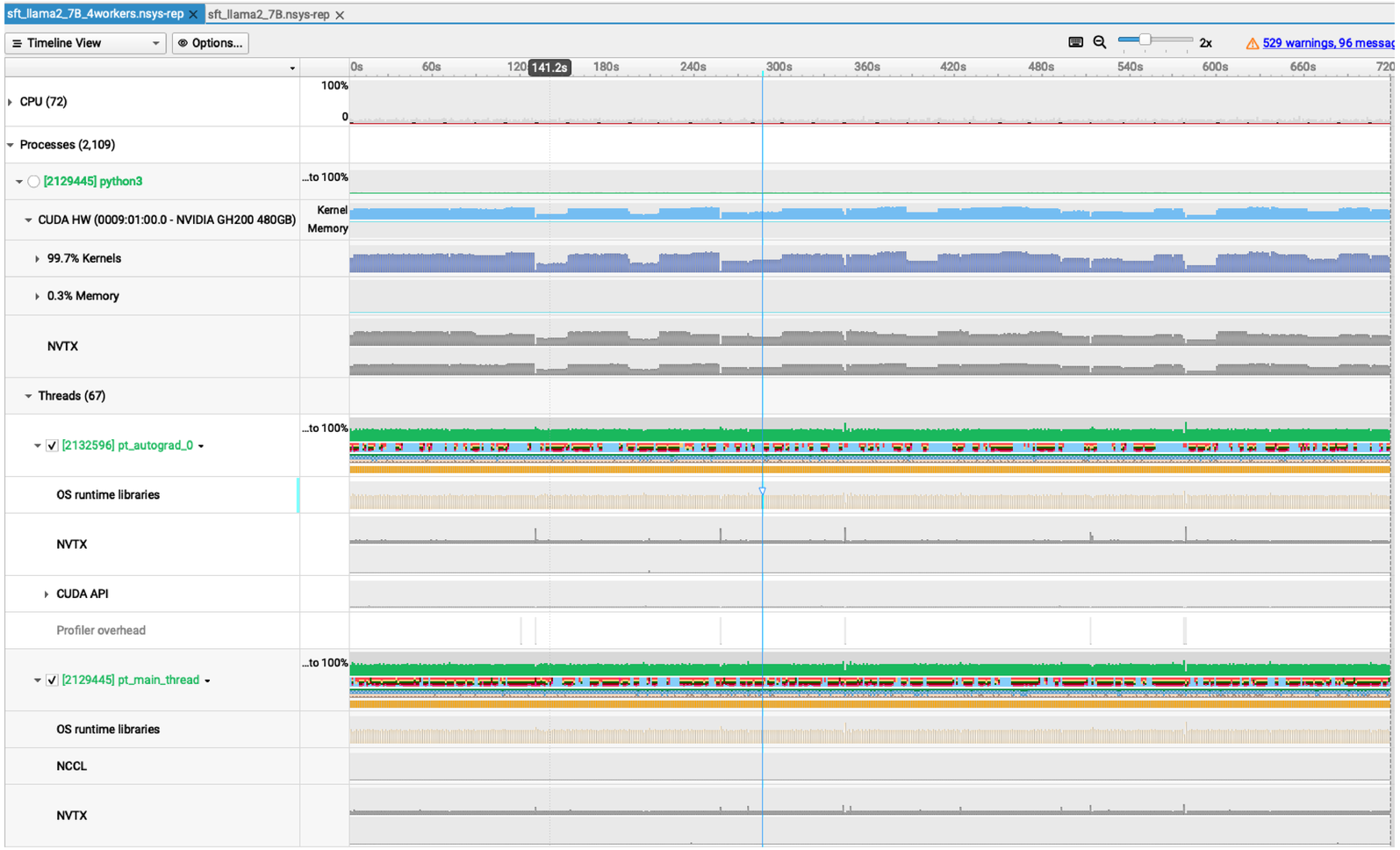
- Green: active computation or execution
- Brown dotted: thread preemption, context switching, waiting for resources
- *pthread_cond_wait*: the application threads are blocked, waiting for a condition variable to be signaled

Interpretation:

- *pthread_cond_wait* might appear while the CPU waits for GPU tasks to finish

Profiling

num_workers=4 (previously 1)

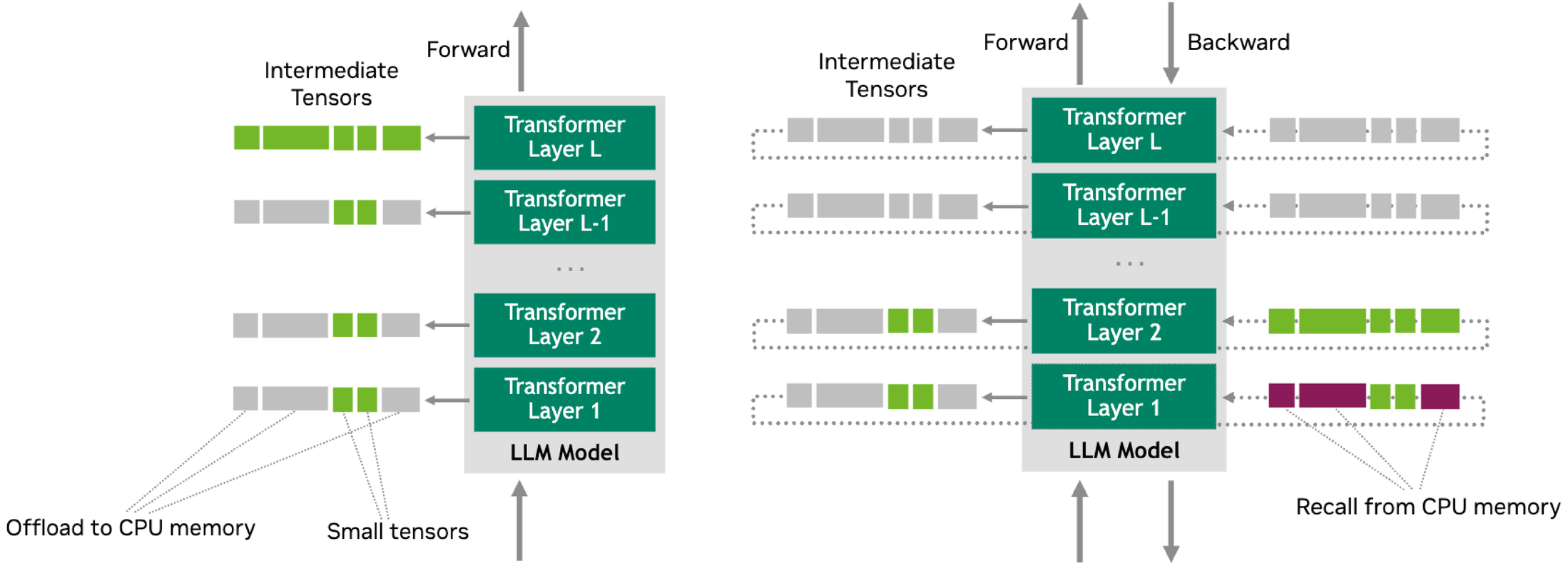


- More consistent utilization with fewer gaps of GPU kernel
- Memory section shows regular patterns of activity -> better pipelining of data to the GPU
- Main thread:
  - More consistent green regions of activity
  - Less time spent in blocking states
  - Better interleaving con computation and data movement
- Next steps:
  - Experiment with num_workers to find optimal setting
  - Increase batch size to fully saturate the GPU

Optimization

Offloading

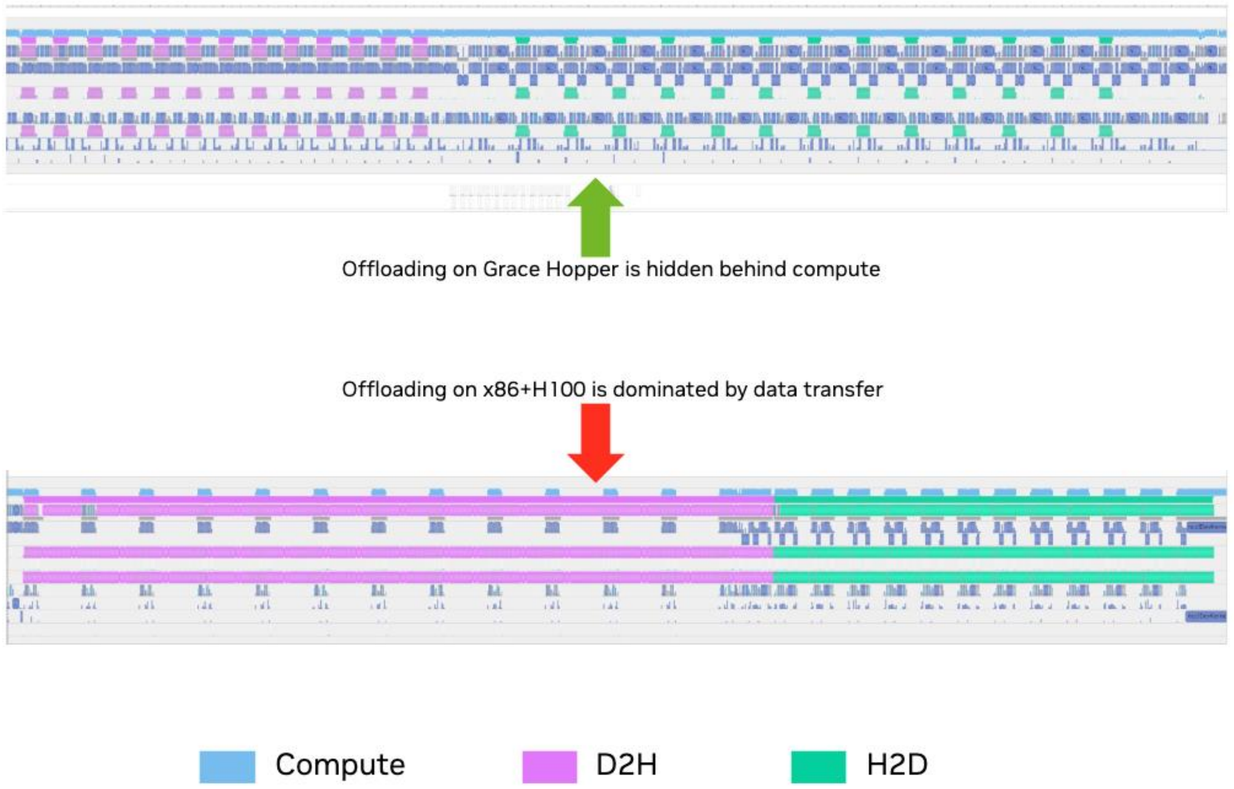
- Transferring parts of the computational workload or memory requirements from GPU to CPU
- **Memory efficiency**: offloading optimizer states, gradients, activations or model parameters to CPUs or system memory extends the effective memory capacity of GPUs

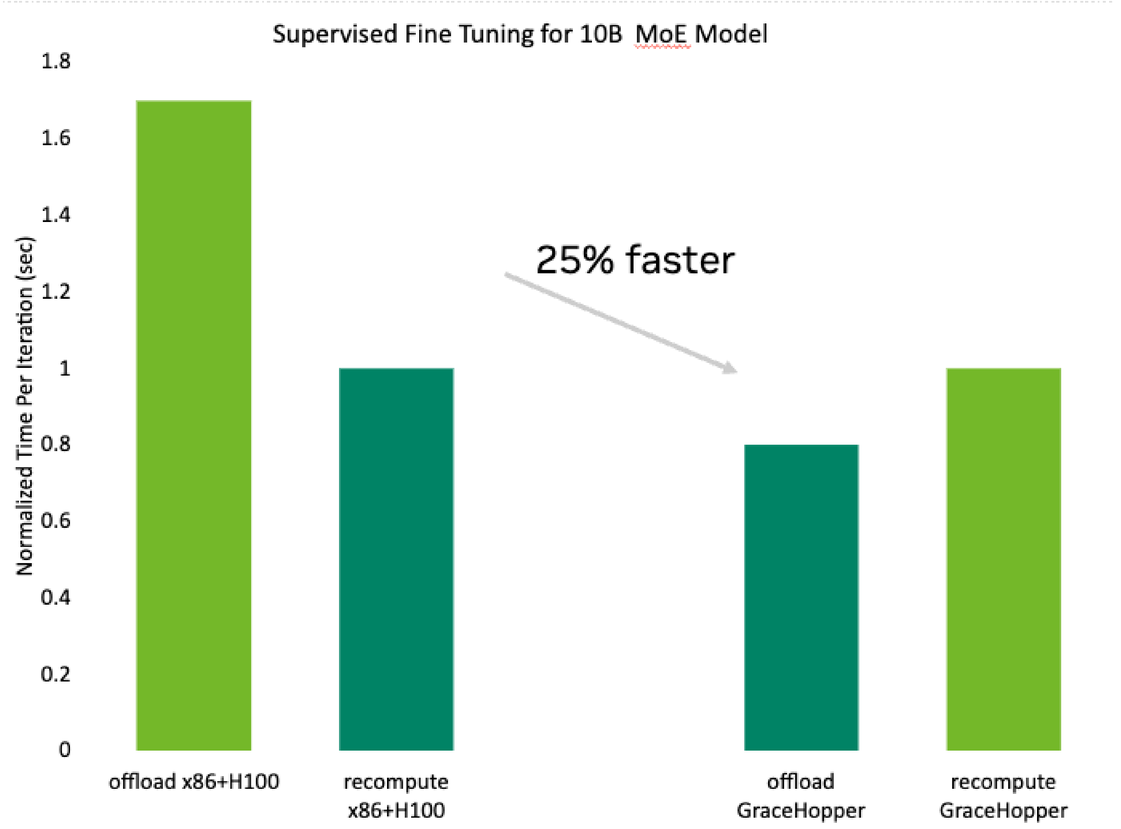


Pictures from "*Grace Hopper Superchip Architecture and Performance Optimizations for AI Applications*" talk, GTC2024

Optimization

Offloading

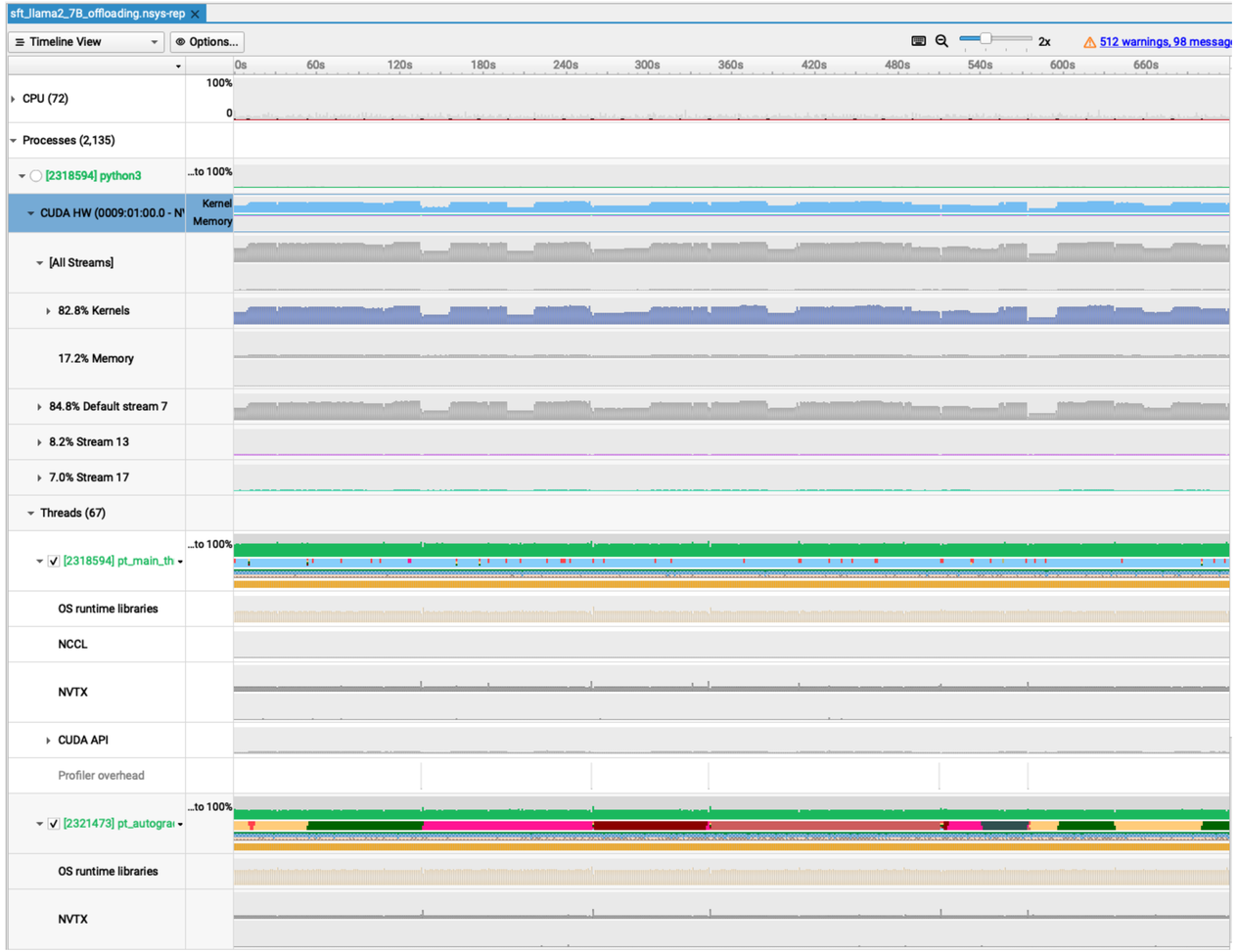




8 NVLink Connected GPUs

Profiling

Offloading



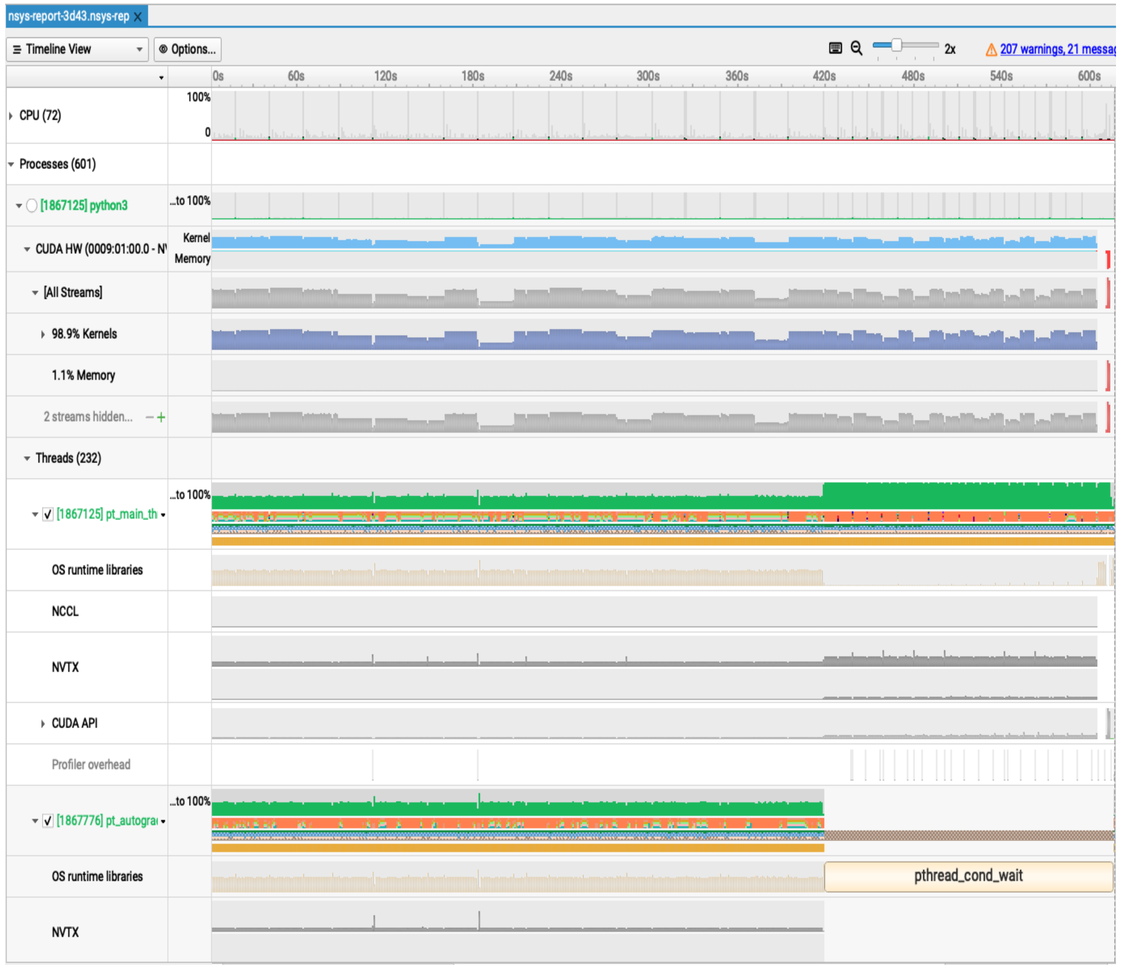
- **Increased CPU activity**:
  - pt_autograd_0 and pt_main_thread threads show more frequent activity, indicating that the CPU is actively involved in managing offloaded activations.
- **Reduced GPU memory usage** in CUDA HW
- **Increased synchronization overhead**:
  - The timeline shows more frequent gaps in GPU kernel execution (blue bars in "CUDA HW"), likely due to increased synchronization between CPU and GPU.

Aspect	Without Offloading	With Offloading
GPU Memory Usage	High	Significantly reduced
GPU Kernel Utilization	More consistent	Gaps due to synchronization
CPU Utilization	Lower	Higher, due to activation management
Synchronization Overhead	Minimal	Increased

Profiling

Unified Memory

- **LoRA Finetuning**



- **No PEFT**



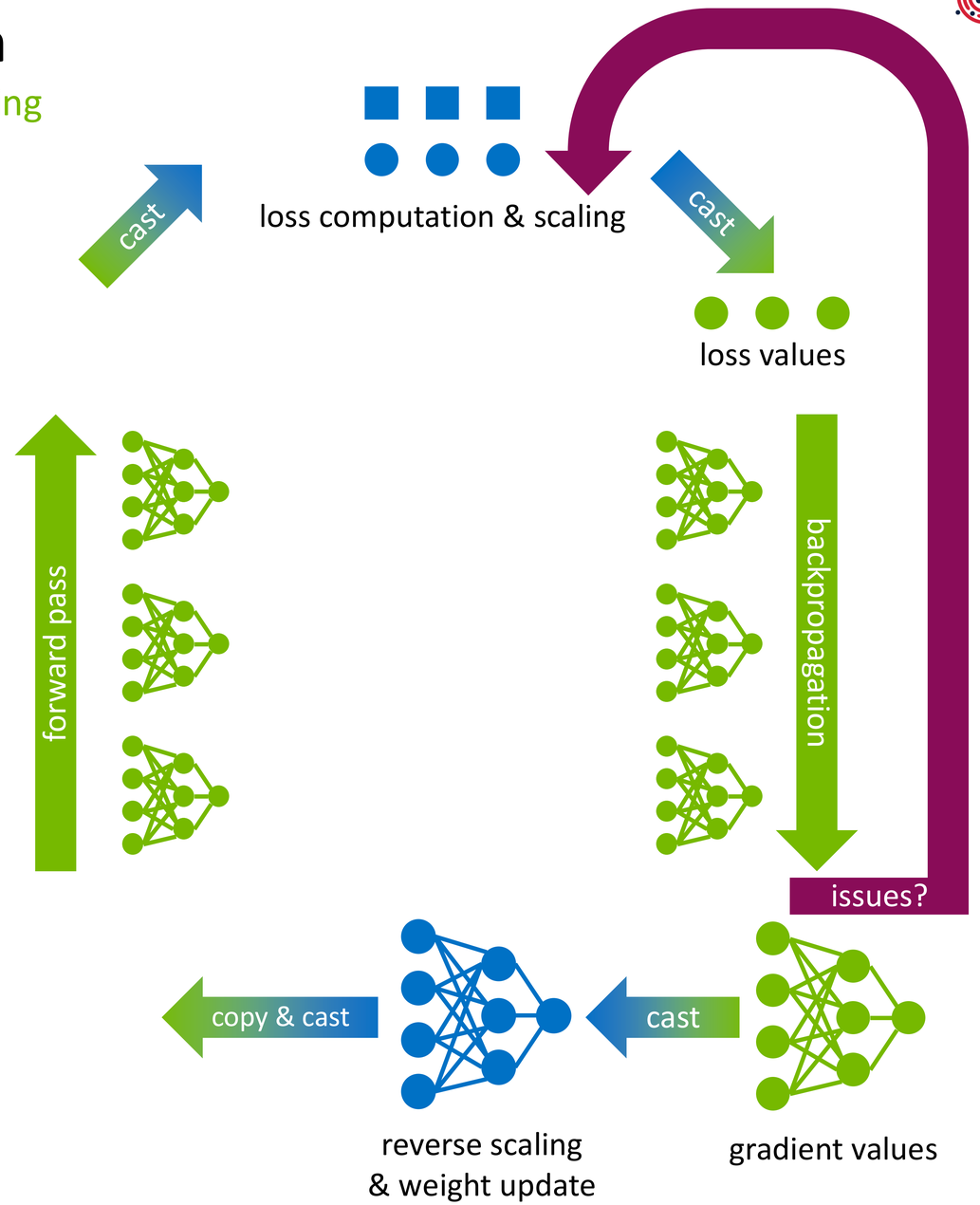
Best Practices for Optimization

Optimization

Mixed Precision Training

- Combines lower-precision and higher precision floating-point formats during model training

- **Increased training speed**: lower-precision formats require fewer computational resources
  - Tensor cores on NVIDIA GPUs (H100 features fourth-generation T.C.) are optimized for mixed precision
- **Memory efficiency**: lower-precision formats reduce memory usage
- **Maintained accuracy**: loss scaling techniques are used to preserve small gradient values and avoid underflow issues



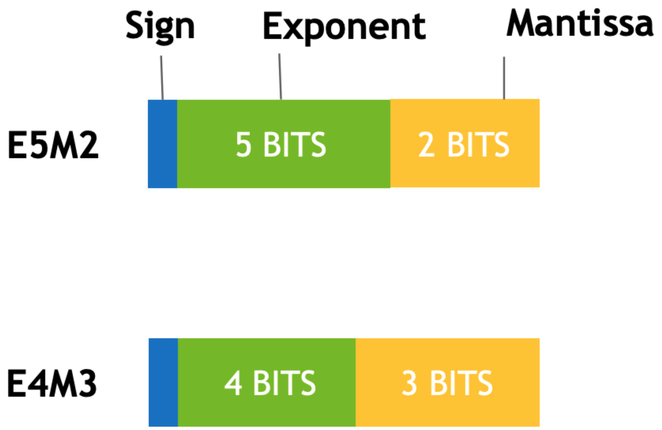
Optimization

FP8 training

- 8-bit floating-point is a low-precision data format, particularly suited for large-scale models like LLMs

- **Improved performance**: FP8 reduces computational overhead, enabling faster forward and backward passes
  - It significantly increases throughput on Hopper GPUs
- **Reduced memory usage**: halves memory requirements compared to FP16/BF16
- **Cost efficiency**: reduces the number of GPUs needed for large-scale training
- **Comparable accuracy**: FP8 achieves accuracy within 1% of BF16 baselines across various tasks

- Two types of FP8:
  - **E5M2**
    - Encoding for infs, Nans, and zeros
    - Dynamic range of 32 powers of 2
    - Precision of 4 samples between powers of 2
  - **E4M3**
    - Encoding for Nans and zeros (No infs)
    - Dynamic range of 18 powers 2
    - Precision of 8 samples between powers of 2



In Conclusion

Summary

- Grace Hopper for Large Language Model training
- Set up of an environment, code and dataset for a PEFT job
- NSIGHT Systems profiling and interpretation
- Overview of optimization techniques

Thank you!

ksevegnani@nvidia.com

References

- **Grace-Hopper Superchip**:
  - https://www.nvidia.com/en-gb/data-center/grace-hopper-superchip/
- **NeMo FW**:
  - https://docs.nvidia.com/nemo-framework/index.html
- **Nsight Systems**:
  - https://developer.nvidia.com/nsight-systems
- **Continued Pretraining**:
  - https://developer.nvidia.com/blog/continued-pretraining-of-state-of-the-art-llms-for-sovereign-ai-and-regulated-industries-with-igenius-and-nvidia-dgx-cloud/
- **Using FP8**:
  - https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html
- **Automatic Mixed Precision**:
  - https://developer.nvidia.com/automatic-mixed-precision
- **NVIDIA AI Technology Center**:
  - https://github.com/NVIDIA-AI-Technology-Center